Hidden Pod Startup Latency

Container Image Pull

Omer Oz · LinkedIn Compute Infra

OmerOz.fyi



Where my perspective comes from



Open source ML infra



Ads ML infra



Compute infra

Massive scale, built for cost efficiency



1B+ users

500K+ servers

25M+ cores

50 pods/host

5M+ pods

100K+ deployments/week

- Thousands of services: stateful, stateless, batch
- Self-hosted bare metal machines with local storage
- Machines stay in service for years → heterogeneous fleet



Spoilers:

3.6× faster pod startup(P99)

1.7× higher pull throughput

Agenda

1. Why does pod startup latency matter
2. The investigation
3. Solution & results
4. Deep dive



Slow pod startups shackle the fleet



Slow pod startups shackle the fleet



- **Laggy**: slow to apply updates

Slow pod startups shackle the fleet



- **Laggy**: slow to apply updates
- **Rigid** ❄: high inertia and friction in deployments, migrations, and scale-ups

Slow pod startups shackle the fleet



- **Laggy**: slow to apply updates
- **Rigid** ❄: high inertia and friction in deployments, migrations, and scale-ups
- **Unreliable**: slow to recover from outages

Slow pod startups shackle the fleet

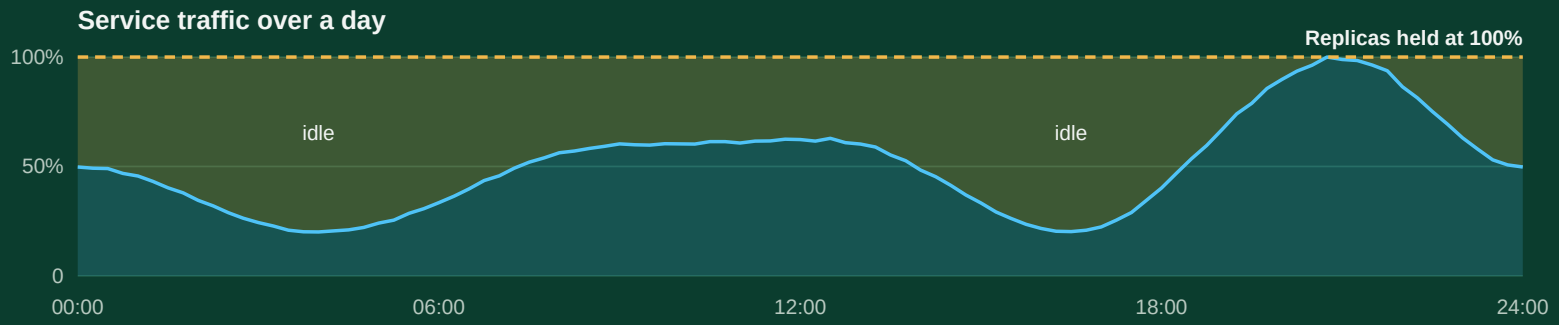


- **Laggy**: slow to apply updates
- **Rigid** ❄: high inertia and friction in deployments, migrations, and scale-ups
- **Unreliable**: slow to recover from outages
- **Wasteful** ☢: overprovisioned replicas and deployment headroom

Slow pod startups shackle the fleet



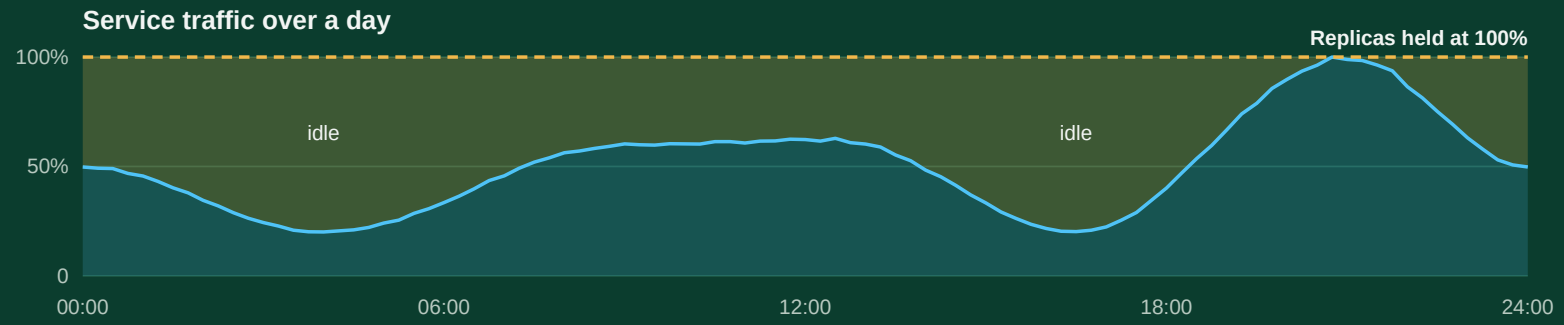
- **Laggy**: slow to apply updates
- **Rigid** ❄: high inertia and friction in deployments, migrations, and scale-ups
- **Unreliable**: slow to recover from outages
- **Wasteful** ☢: overprovisioned replicas and deployment headroom

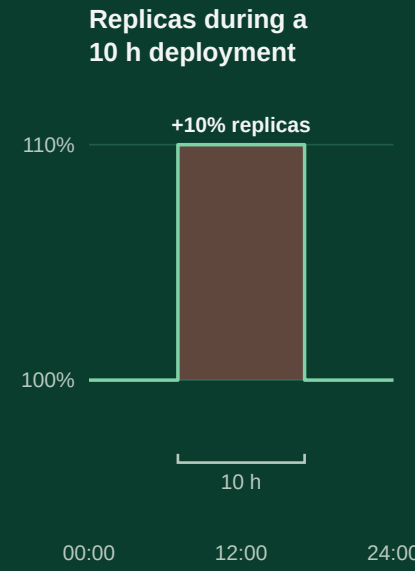


Slow pod startups shackle the fleet



- **Laggy**: slow to apply updates
- **Rigid** ❄: high inertia and friction in deployments, migrations, and scale-ups
- **Unreliable**: slow to recover from outages
- **Wasteful** ☢: overprovisioned replicas and deployment headroom





Fast pod startups set the fleet free



Fast pod startups set the fleet free

- **Responsive**: real-time updates



Fast pod startups set the fleet free

- **Responsive**: real-time updates
- **Fluid**: instant and smooth deployments, migrations, and autoscaling



Fast pod startups set the fleet free

- **Responsive**: real-time updates
- **Fluid**: instant and smooth deployments, migrations, and autoscaling
- **Available**: light-speed recovery from outages



Fast pod startups set the fleet free

- **Responsive**: real-time updates
- **Fluid**: instant and smooth deployments, migrations, and autoscaling
- **Available**: light-speed recovery from outages
- **Efficient**: minimal headroom



Fast pod startups set the fleet free

- **Responsive**: real-time updates
- **Fluid**: instant and smooth deployments, migrations, and autoscaling
- **Available**: light-speed recovery from outages
- **Efficient**: minimal headroom





How did image pulls become the bottleneck?





Knative

How did image pulls become the bottleneck?

ML workloads changed the assumptions:

1. Startup latency: inference autoscaling, training jobs





Knative


KServe

How did image pulls become the bottleneck?

ML workloads changed the assumptions:

1. Startup latency: inference autoscaling, training jobs




Knative


KServe


Kubeflow

How did image pulls become the bottleneck?

ML workloads changed the assumptions:

1. Startup latency: inference autoscaling, training jobs


Knative


KServe


Kubeflow


Kueue

How did image pulls become the bottleneck?

ML workloads changed the assumptions:

1. Startup latency: inference autoscaling, training jobs

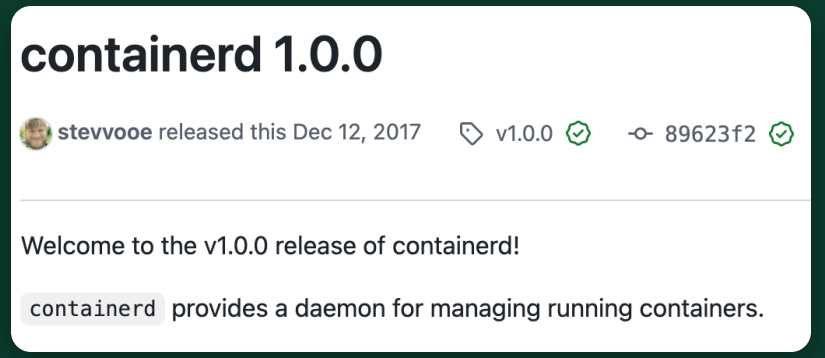

Knative


KServe


Kubeflow


Kueue


KubeRay



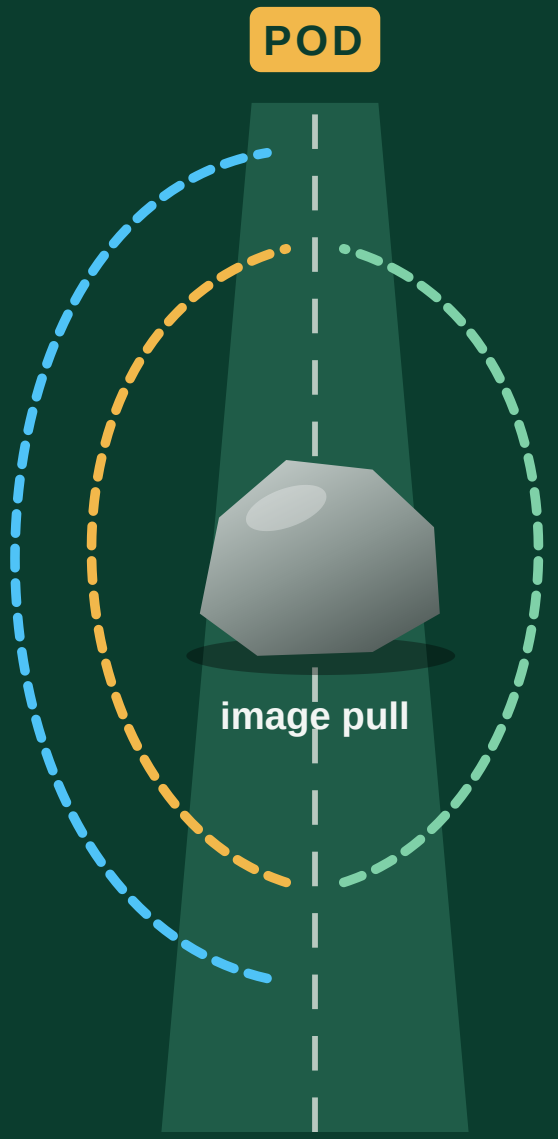
How did image pulls become the bottleneck?

ML workloads changed the assumptions:

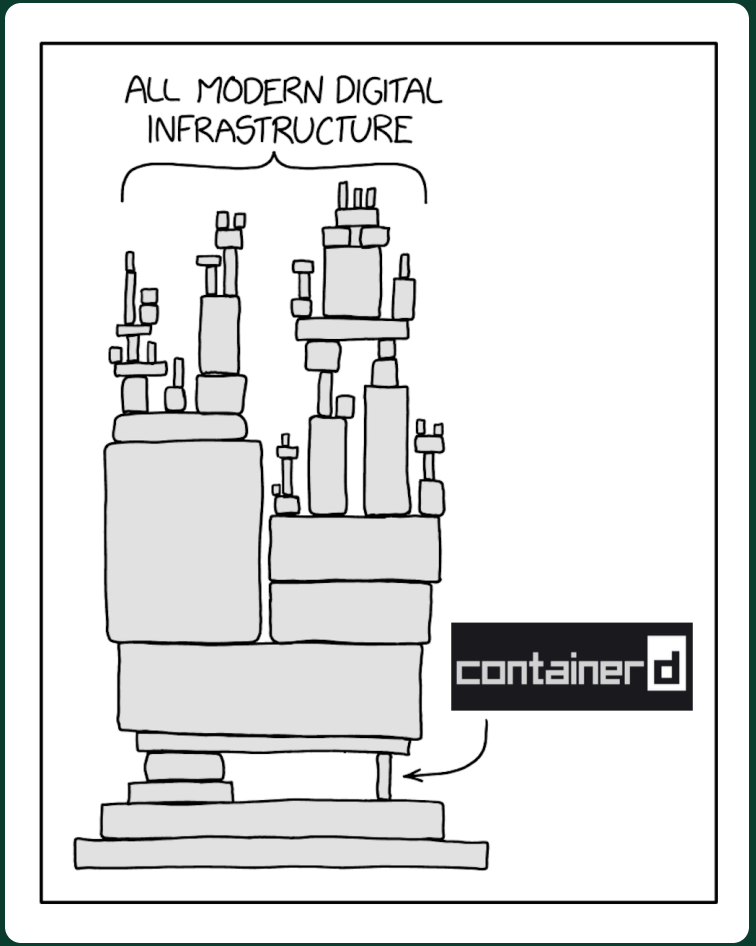
1. Startup latency: inference autoscaling, training jobs
2. Images: 10s of GBs
3. RAM: TBs
4. Network: 100s of Gbps

Complex, expensive workarounds

- **OpenAI**: preload Docker image in machine images [1]
- **Modal**: lazy loading through a multi-tier cache [2]
- **DeepSeek**: lazy loading over RDMA [3]
- **Anthropic**: moves dependencies out of the image; an init sidecar fetches and mounts them at pod start [4]



So basically...



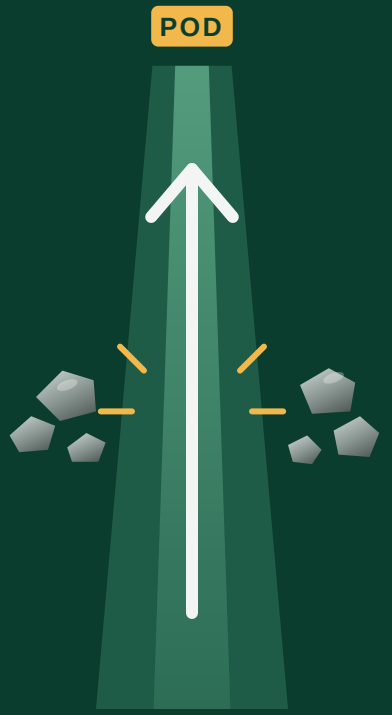
[5]



No more detours

> *Complex systems are intrinsically hazardous systems.*
>
> Richard Cook, *How Complex Systems Fail* (1998)

Let's fix the pull path itself





Artifact distribution leaked into the app layer

- Model weights fetched by serving engines (vLLM, SGLang)
- Runtime downloads
- Init containers
- Persistent volumes
- ...



aka AWS Death Star

28 GB images vs a 20-min SLO

5+ SEV2s

- Flink apps ship large container images
- Slow image pulls breach the SLO

incident-xxx
incident-xxx
incident-xxx
incident-xxx
incident-xxx
incident-xxx
incident-xxx
incident-xxx

Let's watch a 9 GB pull live

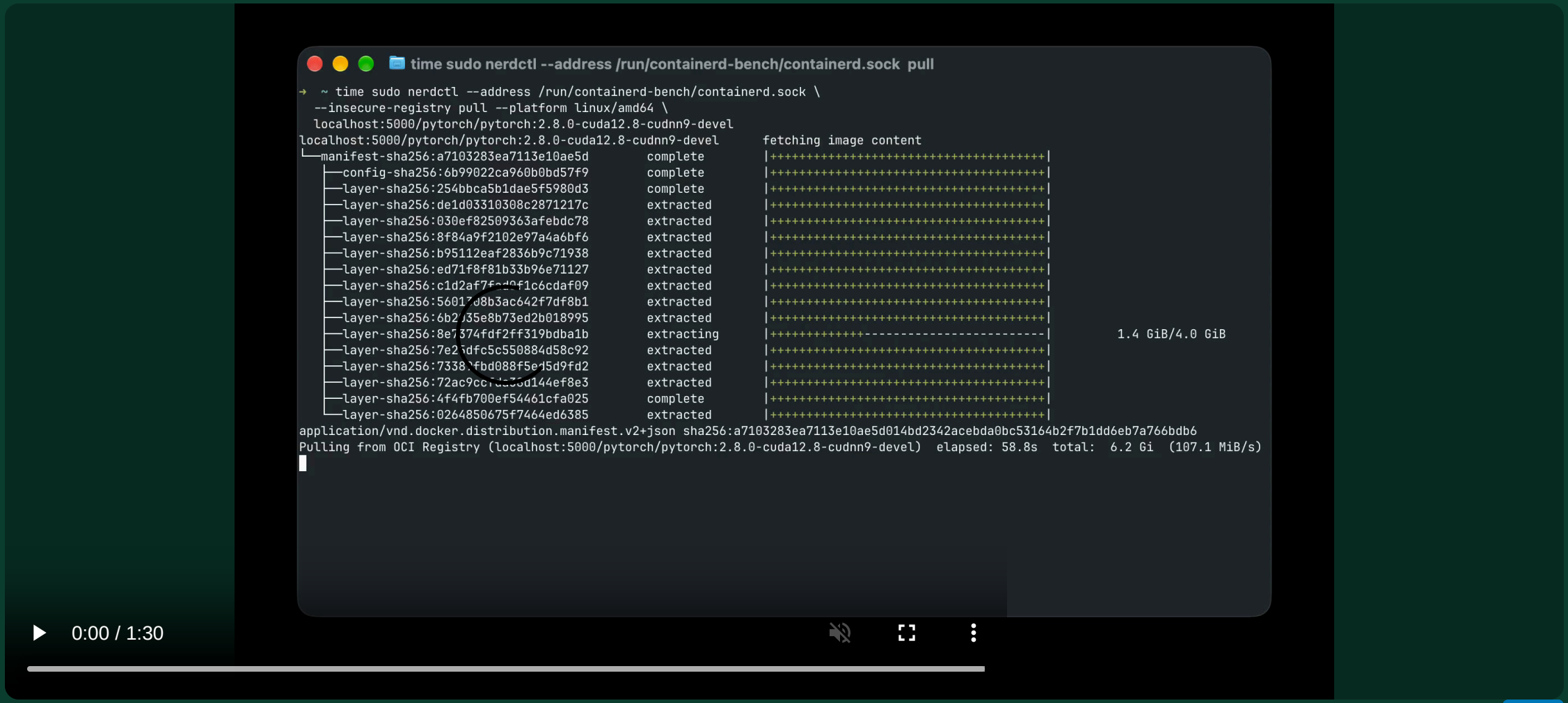




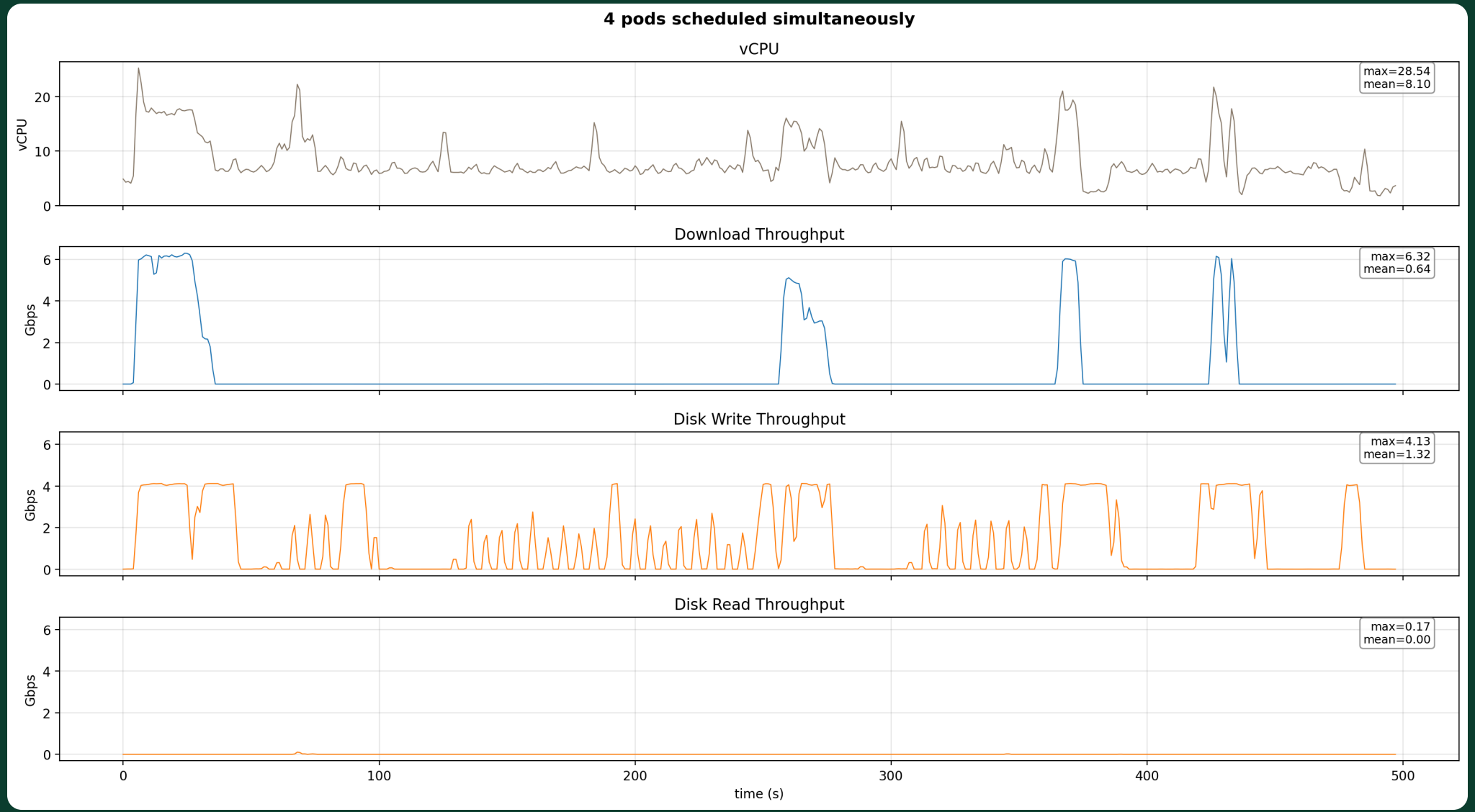
Image pulls are a black box

- containerd logs and documentation provide limited information
- Difficult to test and measure

Investigation

- Profiling
- Benchmarks
- Review containerd codebase



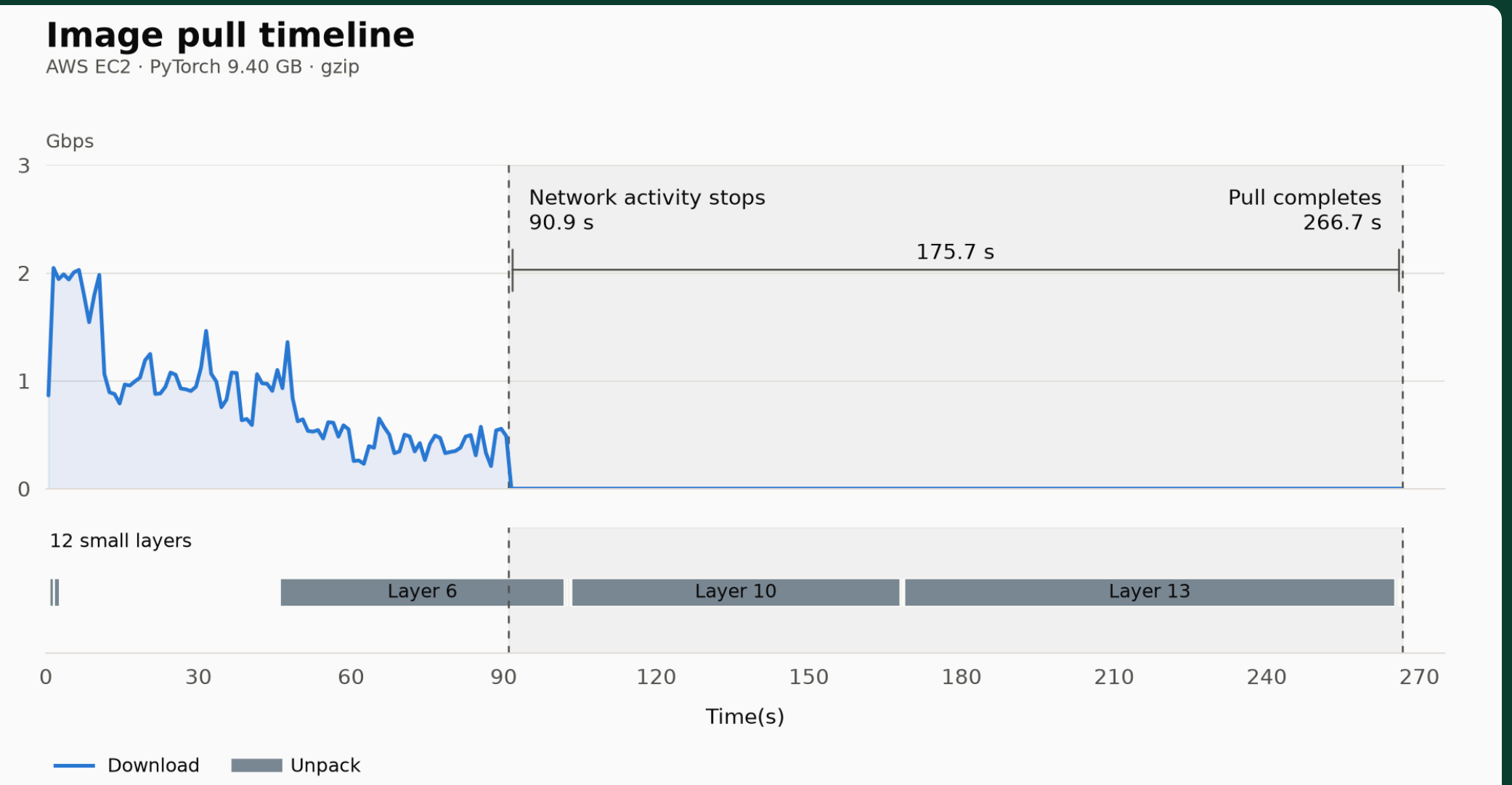

4 pods scheduled simultaneously
vCPU
max=28.54
mean=8.10
vCPU
0
10
20
Download Throughput
max=6.32
mean=0.64
Gbps
0
2
4
6
Disk Write Throughput
max=4.13
mean=1.32
Gbps
0
2
4
6
Disk Read Throughput
max=0.17
mean=0.00
Gbps
0
2
4
6
0
100
200
300
400
500
time (s)

Benchmarks

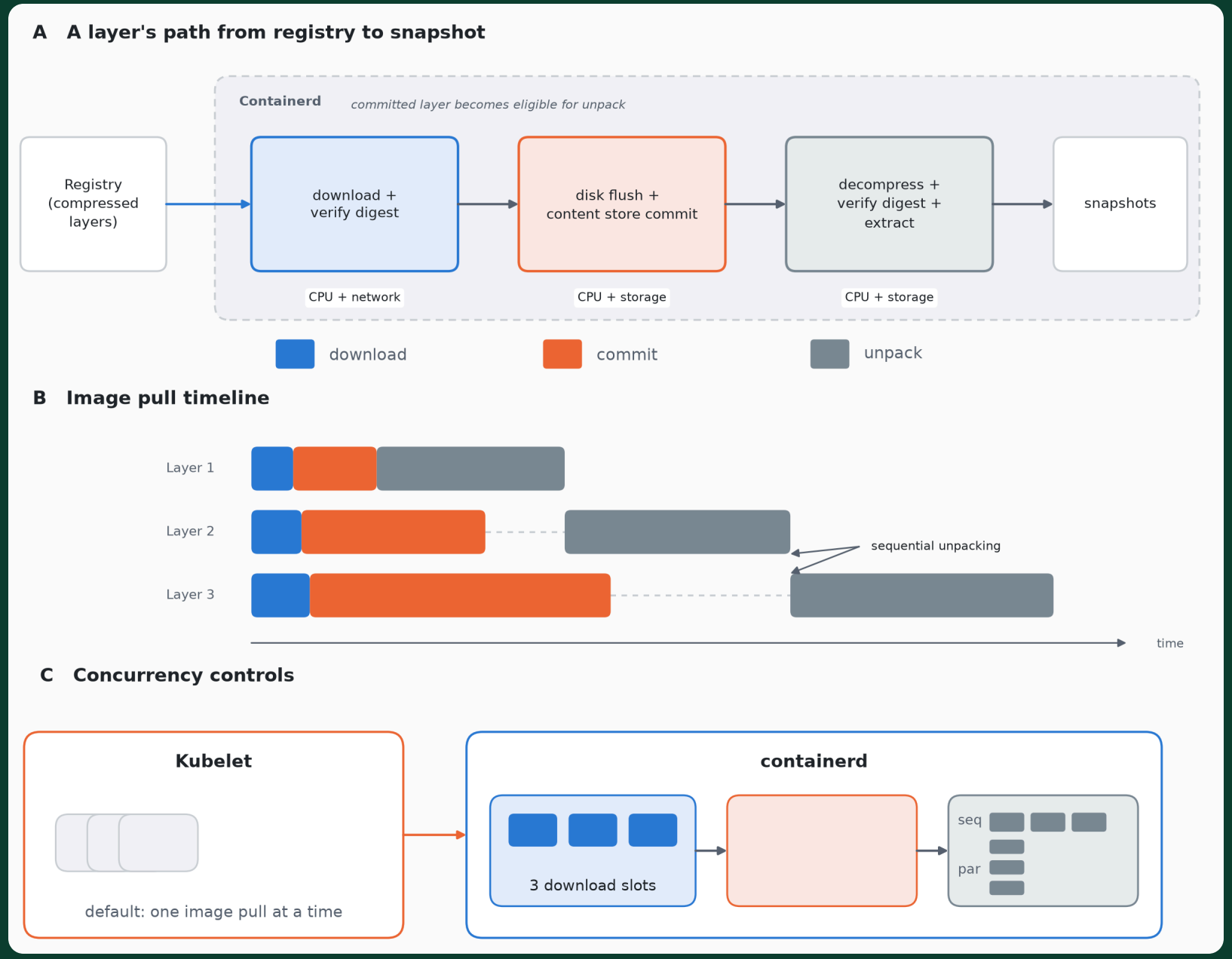
50+ benchmark runs

- Swept kubelet and containerd concurrency settings
- Profiled CPU, network and disk for each run
- Built a mental model of containerd's pull path

```
01,08.csv    01,08.csv
  1,1.csv    1,1,0.csv
1,2,0.csv      1,3.csv
1,3,0.csv    1,4,8.csv
1,6,0.csv    2,2,8.csv
  2,4.csv    2,4,0.csv
2,4,1.csv    2,4,8.csv
2,4,8v2.csv    3,1.csv
3,1,0.csv    3,1,8.csv
3,4,8.csv      4,4.csv
4,4,0.csv    4,4,8.csv
6,4,0.csv     30,1.csv
30,08.csv          ...
```


Image pull timeline
AWS EC2 · PyTorch 9.40 GB · gzip
Gbps
3
2
1
0
Network activity stops
90.9 s
Pull completes
266.7 s
175.7 s
12 small layers
Layer 6
Layer 10
Layer 13
0
30
60
90
120
150
180
210
240
270
Time(s)
Download
Unpack

[6]

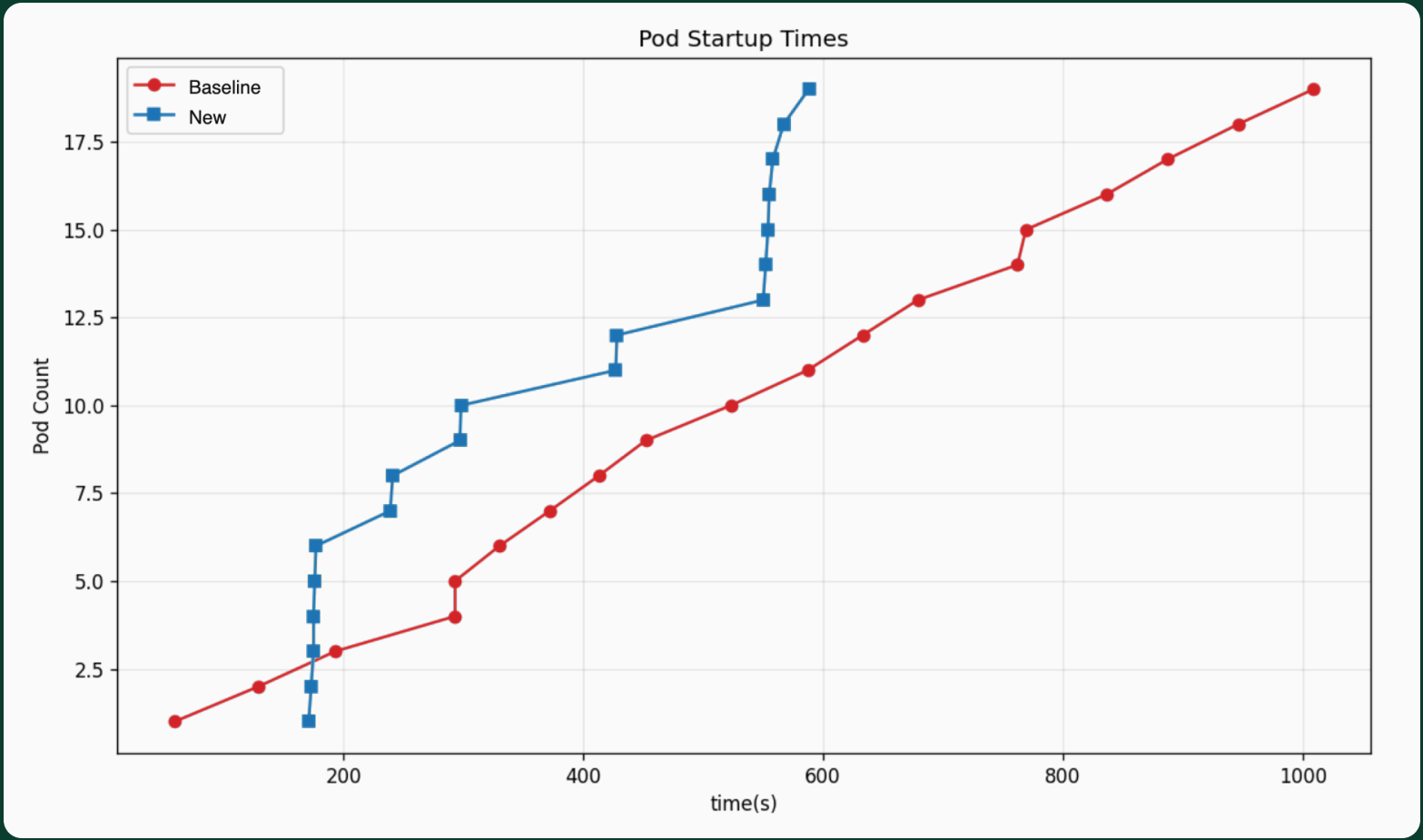

A A layer's path from registry to snapshot
Containerd
committed layer becomes eligible for unpack
Registry (compressed layers)
download + verify digest
disk flush + content store commit
decompress + verify digest + extract
snapshots
CPU + network
CPU + storage
CPU + storage
download
commit
unpack
B Image pull timeline
Layer 1
Layer 2
Layer 3
sequential unpacking
time
C Concurrency controls
Kubelet
default: one image pull at a time
containerd
3 download slots
seq
par

[6]

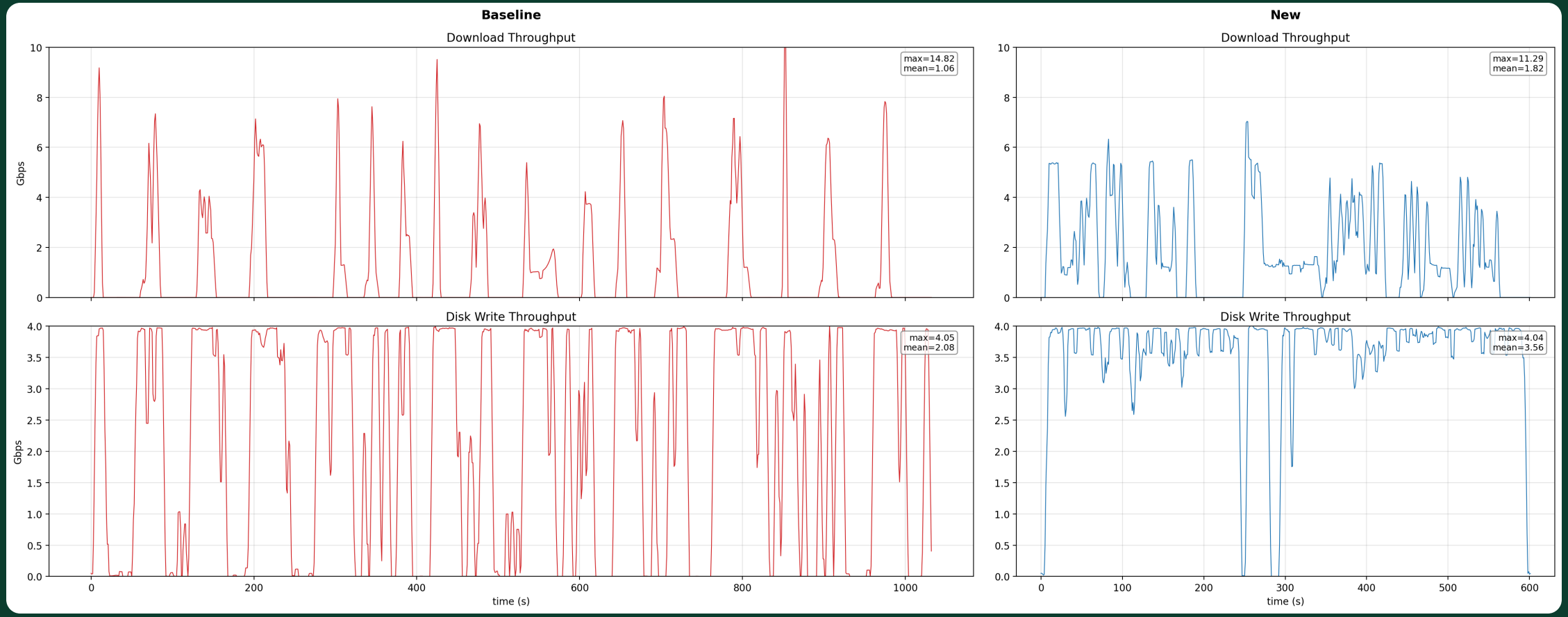
Solution: tune pull concurrency

Setting	Baseline	New	Effect
kubelet `maxParallelImagePulls`	1	3	Unpack images in parallel
containerd `max_concurrent_downloads`	3	1	Limit download bursts

Results: 19 pods startup timeline



Results: pull throughput ↑71%



Results: pull metrics

Latency: ↓**72%** P99, ↓**75%** P50

Flink apps
Single tenant

30K hosts
1.5M cores
11 PB RAM



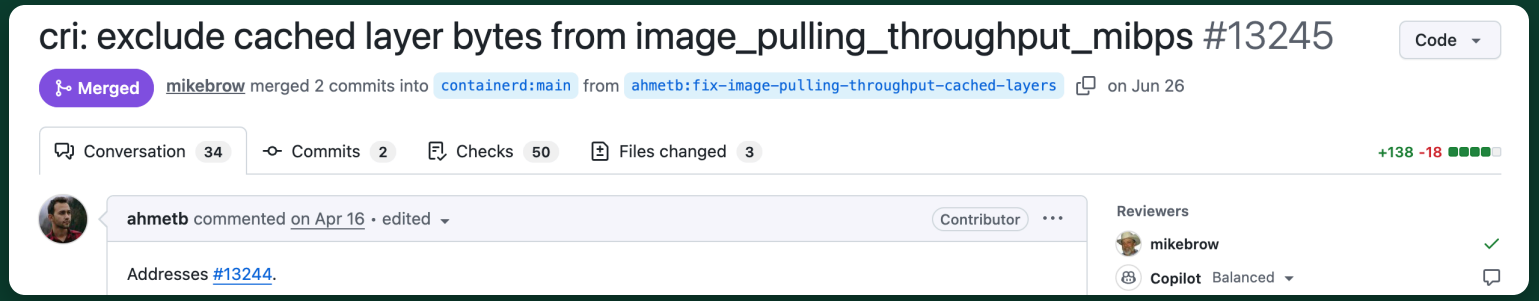
Stateless apps
Multi tenant



Observability

Host level

- kubelet `kubelet_image_pull_duration_seconds`
- containerd `image_pulling_throughput_mibps` overcounted cached layers, **we fixed it upstream**



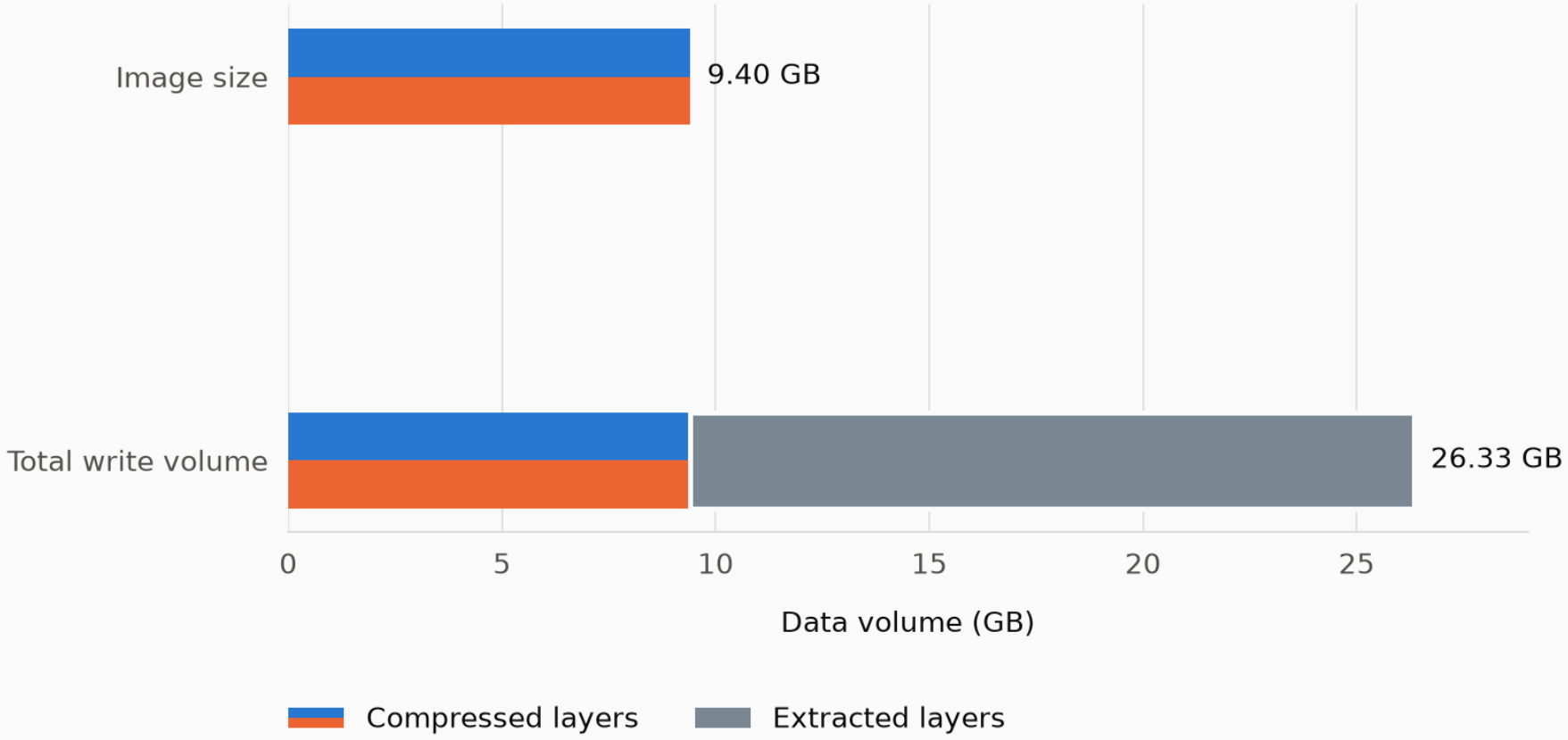
Our own granular metrics

- Pull metrics built from Kubernetes events

Unpack deep dive

Storage amplification during image pulls

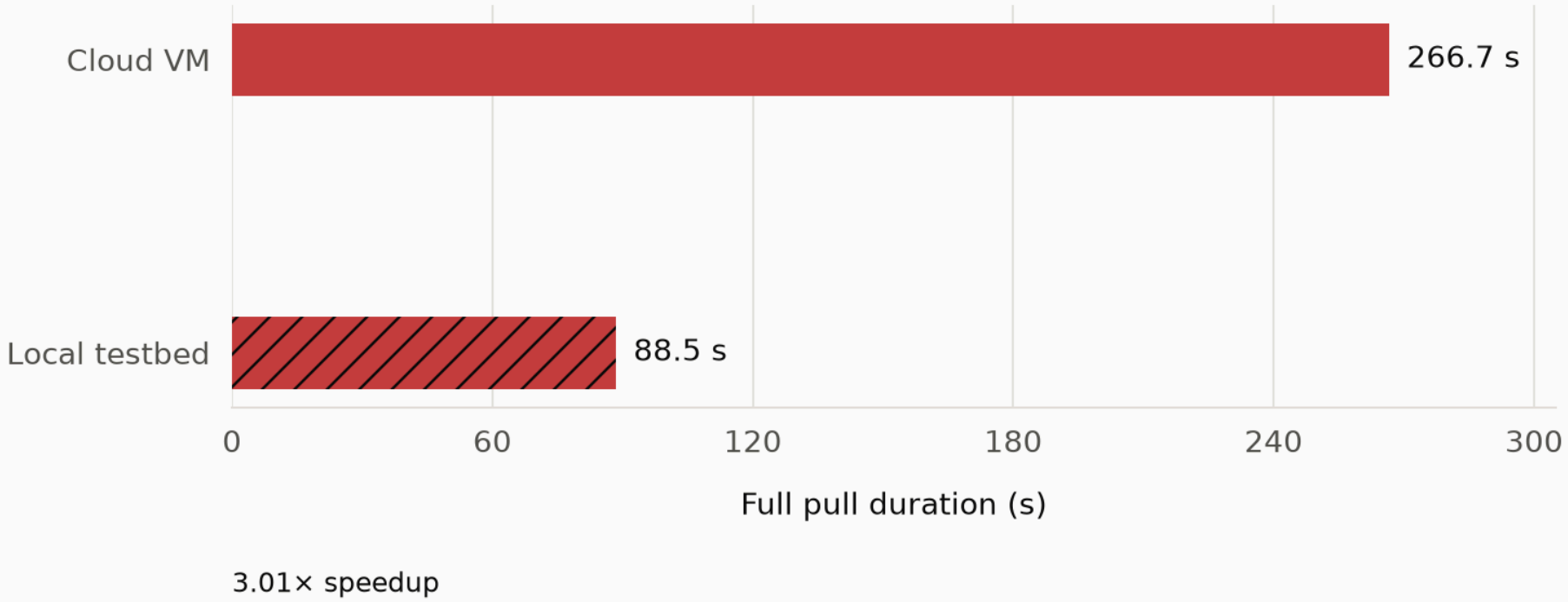
PyTorch 9.4 GB



[6]

The cost of the slow disk

PyTorch 9.4 GB · gzip · sequential unpack



3.01× speedup

[6]

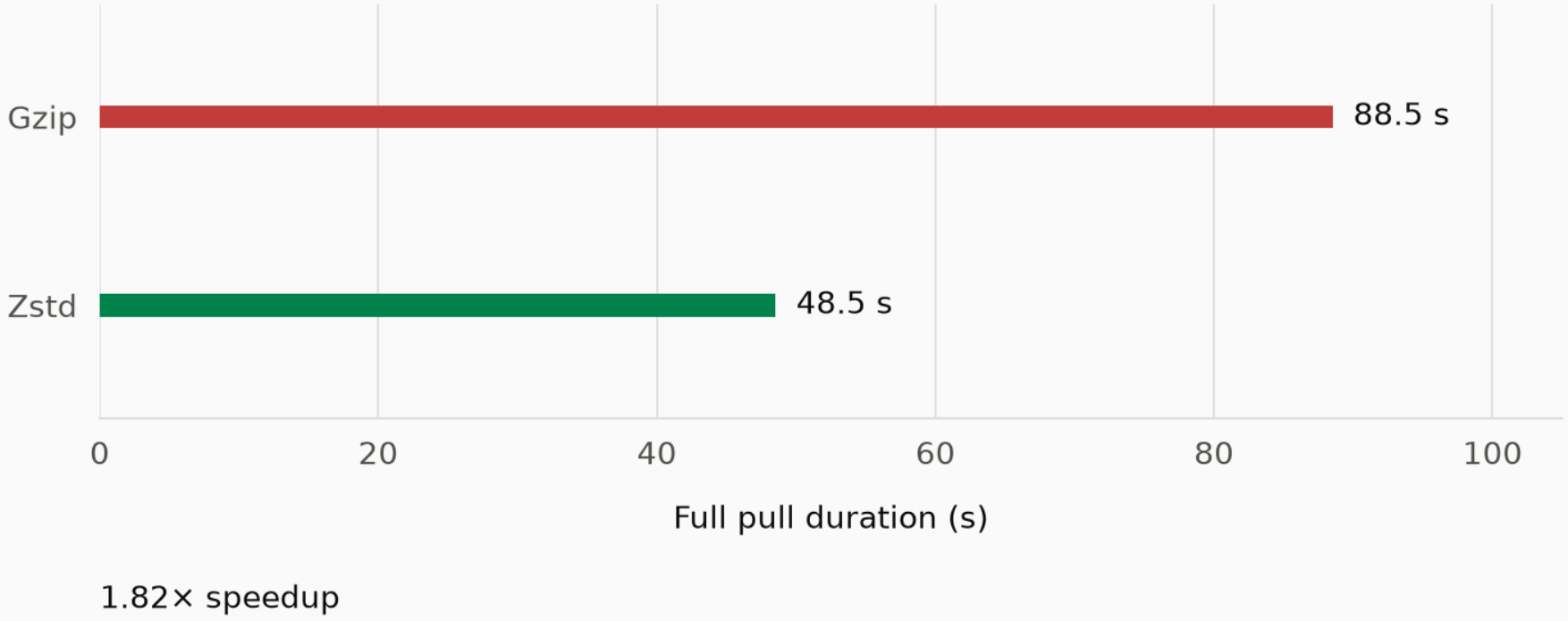
You should invest in NVMe



WD 4TB NVMe	44 Gbps	$500
WD 4TB SATA	6 Gbps	$880
SG 4TB HDD	1.7 Gbps	$266
Cloud Block Storage	1 Gbps	-

The cost of gzip

PyTorch · sequential unpack

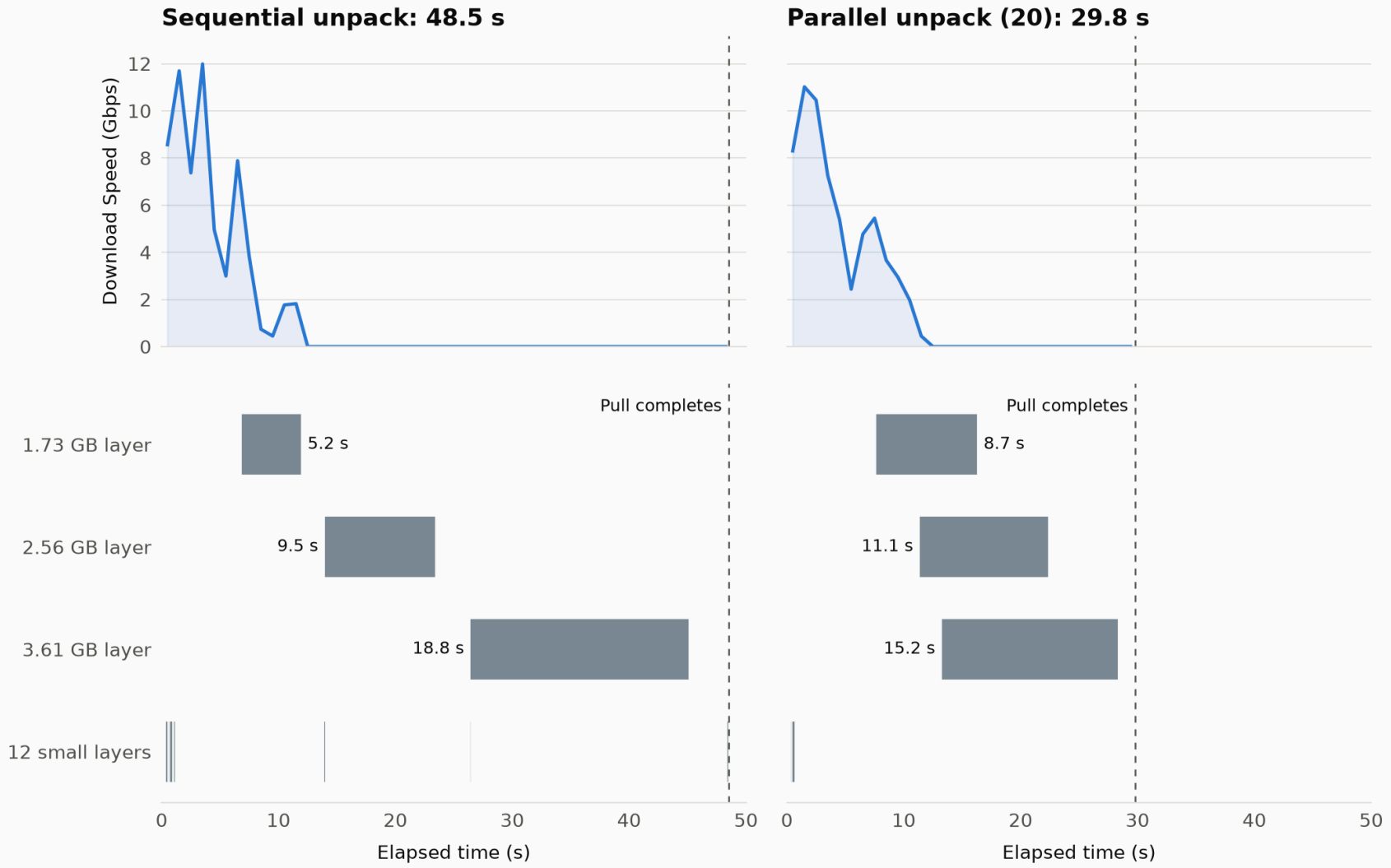


1.82× speedup

[6]

The cost of sequential unpacking

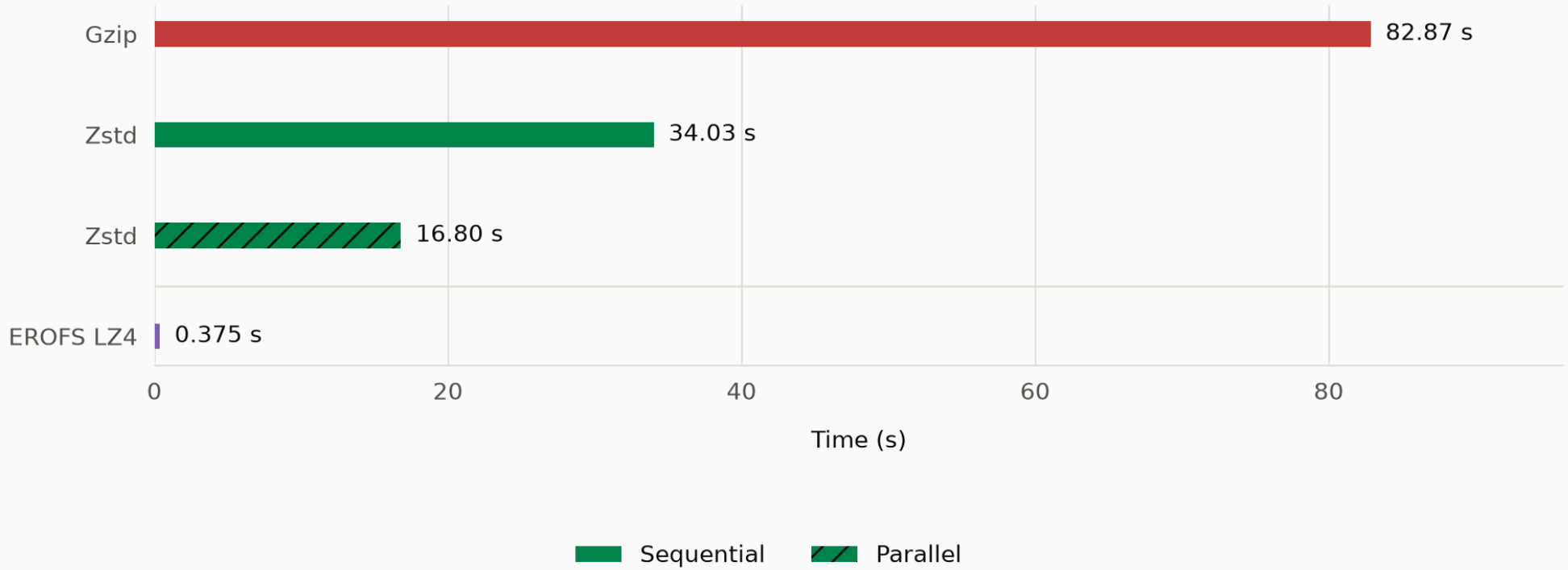
PyTorch · zstd



[6]

The cost of eager unpacking

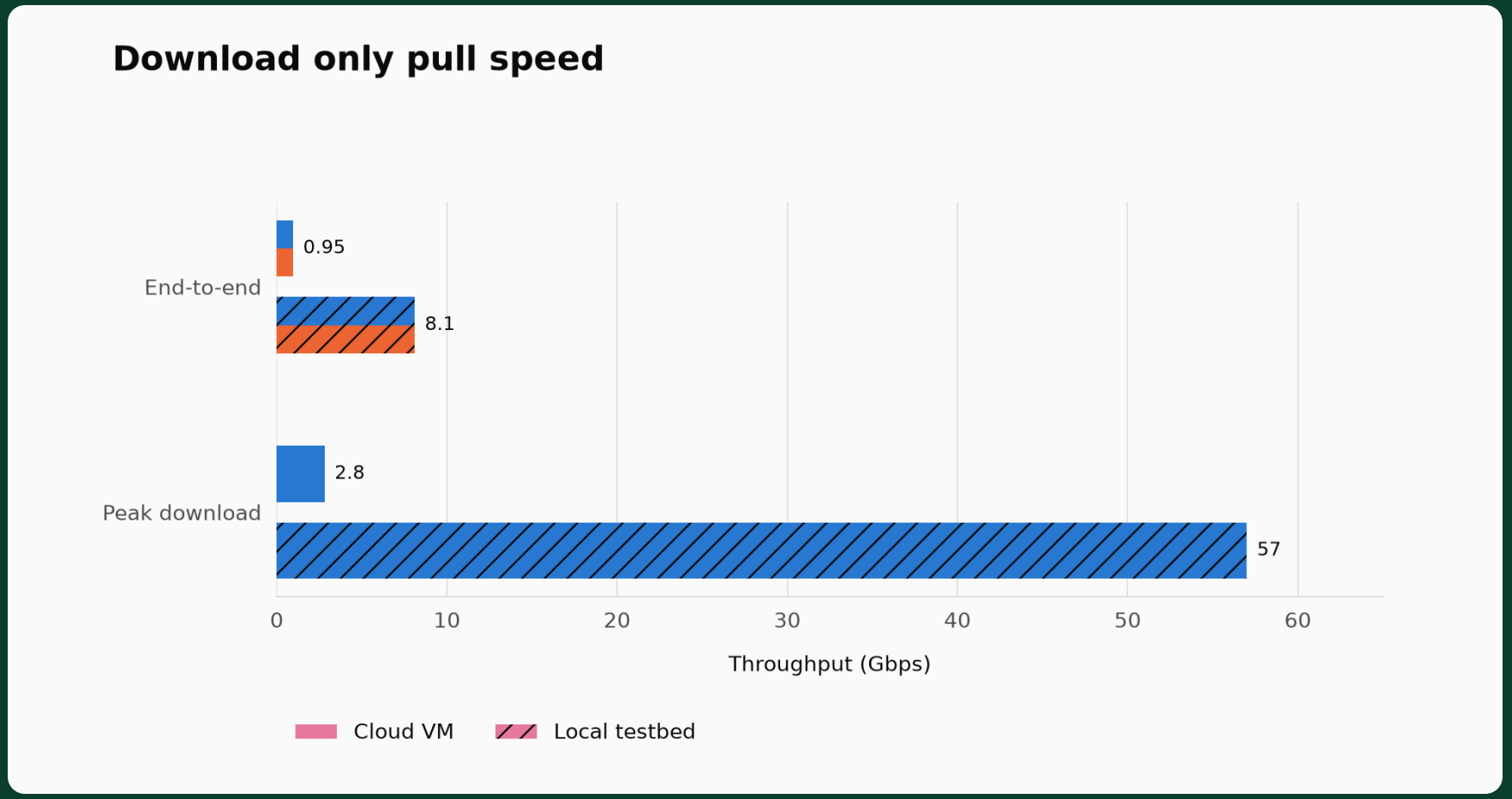
PyTorch image · unpack only



[6]

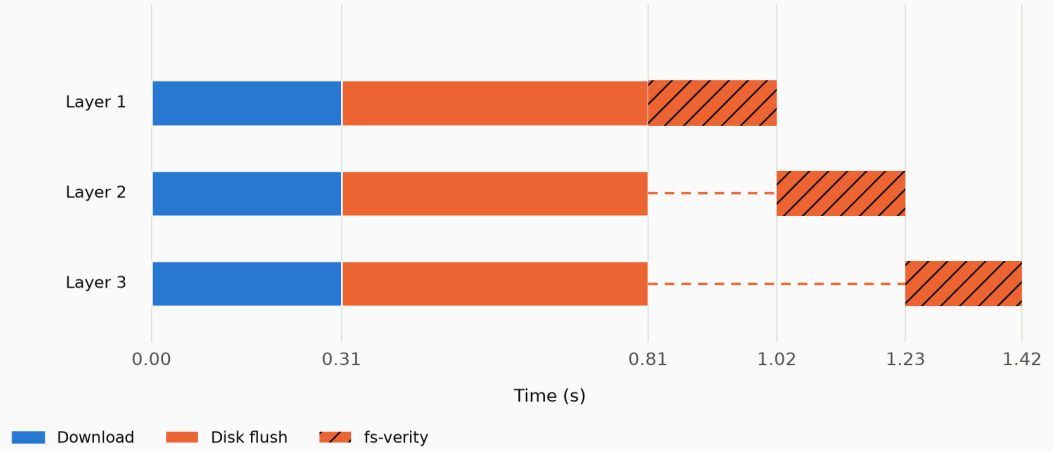
Download deep dive

Download deep dive: container registry



[6]

The cost of fs-verity serialization



content: remove fs-verity serialization point #14442

Open omerXfaruq wants to merge 3 commits into containerd:main from omerXfaruq:fix/content-verity-lock

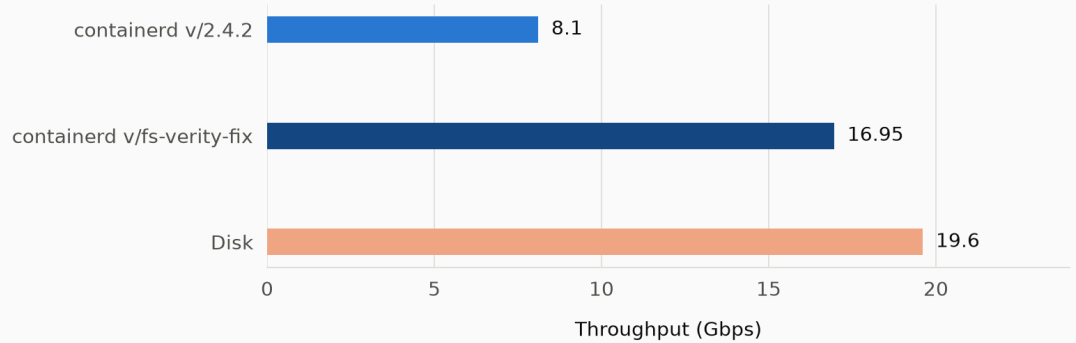
Conversation 6 Commits 3 Checks 53 Files changed 6

omerXfaruq commented last week • edited

Summary

Move expensive content preparation, including fs-verity setup, outside the metadata write transaction.

Removing fs-verity serialization



Scope	Download bottleneck	Gbps
Per layer	HTTP	50
Per layer	SHA-256 + fs-verity	8.6
Per image	disk	write throughput

[6]

Download deep dive: P2P



Thank you

The Hidden Critical Path: An Incremental Analysis of Container Image Pull Bottlenecks in containerd

Omer Oz, Oct 2026, v.1

Questions, feedbacks, slides and deep dives

kbcn.OmerOz.fyi

Omer Oz · LinkedIn Compute Infra

OmerOz.fyi



Hidden Pod Startup Latency: Container Image Pull

OpenAI, Scaling Kubernetes to 2,500 nodes, 2018 [1]
Modal, Building fast filesystems for VMs, 2026 [2]
DeepSeek, DeepSeek Elastic Compute (DSec), arXiv 2609.22978, 2026 [3]
A. Agrawal (Anthropic), Docker Was Too Slow, So We Replaced It: Nix in Production, SCALE 22x, 2025 [4]
Adapted from xkcd 2347, Dependency, Randall Munroe (CC BY-NC 2.5) [5]
Omer Oz, The Hidden Critical Path: An Incremental Analysis of Container Image Pull Bottlenecks in containerd, 2026 [6]